

RTOS CE
开发指南

版本号: 1.3
发布日期: 2025.6.30

深圳易新泰科技有限公司YiXinTai

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.10.30	SWC	1. 初版
1.1	2022.10.26	AWA0916	1. 新增 R128/XR875
1.2	2025.5.15	XAA0345	1. 新增 F133 2. 新增 hwcrypto 的配置方法 3. 新增测试命令
1.3	2025.6.30	XAA0345	1. 新增 F135

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
- 1.1 文档简介 . . . 1
- 1.2 目标读者 . . . 1
- 1.3 适用范围 . . . 1
- 1.4 文档约定 . . . 1
  - 1.4.1 标志说明 . . . 1
  - 1.4.2 地址与数据描述方法约定 . . . 2
  - 1.4.3 数值单位约定 . . . 2
  - 1.4.4 硬件术语 . . . 2
  - 1.4.5 软件术语 . . . 3

2 模块介绍 **4**
- 2.1 模块功能介绍 . . . 4
- 2.2 模块配置介绍 . . . 4
- 2.3 源码结构介绍 . . . 4

3 模块接口说明 **6**
- 3.1 CE 初始化接口 . . . 6
- 3.2 CE 去初始化接口 . . . 6
- 3.3 AES 算法加解密接口 . . . 7
- 3.4 HASH 算法运算接口 . . . 7
- 3.5 RSA 算法运算接口 . . . 8
- 3.6 RNG 算法运算接口 . . . 8

4 hwcrypto 设备 **10**
- 4.1 hwcrypto 配置介绍 . . . 10
- 4.2 symmetric 设备管理接口 . . . 10
  - 4.2.1 创建对称加解密上下文 . . . 10
  - 4.2.2 设置加解密密钥 . . . 11
  - 4.2.3 设置对称加解密初始化向量 . . . 11
  - 4.2.4 加解密接口 . . . 11

5 模块使用范例 **13**

6 FAQ **15**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文将介绍 CE 驱动的设计结构、流程、API 接口。主要包含 CE 接口部分和算法处理部分。

1.2 目标读者

硬件底层设计人员，驱动编写、维护人员，应用开发人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
V833	Melis	drivers/hal/source/ce/*
R328	FreeRTOS	drivers/hal/source/ce/*
R128	FreeRTOS	drivers/hal/source/ce/*
XR875	FreeRTOS	drivers/hal/source/ce/*
F133	Melis	drivers/hal/source/ce/*
F135	Melis	drivers/hal/source/ce/*

1.4 文档约定

1.4.1 标志说明

⚠ 注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000
	1 M	1 000 000
	1 G	1 000 000 000

1.4.4 硬件术语

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 1-4: 硬件术语

术语	解释说明
CE	Crypto Engine，算法引擎，以前称作 SS
TRNG	True Random Number Generator，真随机数发生器

1.4.5 软件术语

表 1-5: 软件术语

术语	解释说明
AES	Advanced Encryption Standard，高级加密标准
CBC	Cipher Block Chaining mode，加密块链模式
ECB	Electronic Code Book mode，电子密码本模式
MD5	Message Digest Algorithm 5，消息摘要算法第五版
SHA	Secure Hash Algorithm，安全散列算法
RSA	一种公钥加密算法

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

CE 模块主要支持对称算法、非对称算法、摘要算法进行数据的加密和解密功能。

CE 模块主要支持的算法如下：

- AES 算法 ECB/CBC/CTR/CTS/OFB/CFB/CBC-MAC/XTS 等模式.
- HASH 算法 MD5/SHA1/SHA224/SHA256/SHA384/SHA512/HMAC-SHA1/HMAC-SHA256.
- 非对称算法 RSA512/1024/2048/3072/4096.

2.2 模块配置介绍

其 menuconfig 的配置如下：

打开配置CONFIG_DRIVERS_CE，支持 CE 驱动打开CONFIG_HAL_TEST_CE即可在命令行执行对应命令测试对应算法

```
Kernel Setup --->
 Drivers Setup --->
  SoC HAL Drivers --->
   CE devices --->
    [*] enable ce driver
    [*] enbale ce hal APIs Test command
```

2.3 源码结构介绍

CE 驱动位于 source/drivers/hal/source/ce/目录下。

```
hal/
├── source
│   ├── ce
│   │   ├── ce_common.c          # CE公用操作接口函数文件
│   │   ├── ce_common.h          # CE操作接口函数头文件
│   │   ├── hal_ce.c             # CE底层驱动文件
│   │   ├── hal_ce.h             # CE底层驱动头文件
│   │   ├── Makefile
│   │   └── platform.h           # 平台配置头文件
│   ├── platform
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
├── ce_sun8iw18.h          # 具体的平台配置头文件
│   └── ce_sun8iw19.h      # 具体的平台配置头文件
│   └── ce_sun20iw2.h      # 具体的平台配置头文件
└── include/hal
    ├── sunxi_hal_ce.h     # CE公用操作接口函数头文件
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

CE 提供的接口列表如下：

```
int sunxi_ce_init(void);
int sunxi_ce_uninit(void);
int do_aes_crypto(crypto_aes_req_ctx_t *req_ctx);
int do_hash_crypto(crypto_hash_req_ctx_t *req_ctx);
int do_rsa_crypto(crypto_rsa_req_ctx_t *req_ctx);
int do_rng_gen(crypto_rng_req_ctx_t *req_ctx);
```

3.1 CE 初始化接口

- 原型：int sunxi_ce_init(void)
- 作用：CE 模块初始化，主要申请中断、clk 初始化等
- 参数：
  - 无
- 返回：
  - 0：成功
  - 负数：失败

3.2 CE 去初始化接口

- 原型：int sunxi_ce_uninit(void)
- 作用：CE 模块去初始化，主要注销中断等
- 参数：
  - 无
- 返回值：
  - 0：成功
  - 负数：失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 AES 算法加解密接口

- 原型：int do_aes_crypto(crypto_aes_req_ctx_t *req_ctx)
- 作用: 主要实现对 AES 算法进行加解密
- 参数：
  - req_ctx: 为 AES 算法上下文的结构体
- 返回值：
  - 0：成功
  - 负数：失败
- 上下文数据结构：

```
typedef struct {
    uint8_t *src_buffer;
    uint32_t src_length;
    uint8_t *dst_buffer;
    uint32_t dst_length;
    uint8_t *iv;
    uint8_t *iv_next;
    uint8_t *key;
    uint32_t key_length;
    __aligned(CACHELINE_LEN) uint8_t padding[AES_BLOCK_SIZE];
    uint32_t padding_len;
    uint32_t dir;  /*0--加密，1--解密*/
    uint32_t mode;  /*AES算法的模式*/
    uint32_t bitwidth;
} crypto_aes_req_ctx_t;
```

3.4 HASH 算法运算接口

- 原型：int do_hash_crypto(crypto_hash_req_ctx_t *req_ctx)
- 作用: 主要实现对 HASH 算法进行运算
- 参数：
  - req_ctx: 为 HASH 算法上下文的结构体
- 返回值：
  - 0：成功
  - 负数：失败
- 上下文数据结构：

```
typedef struct {
    uint8_t *src_buffer;
    uint32_t src_length;
    uint8_t *dst_buffer;
    uint32_t dst_length;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    __aligned(CACHELINE_LEN) uint8_t md[SHA_MAX_DIGEST_SIZE];
    uint32_t md_size;
    __aligned(CACHELINE_LEN) uint8_t padding[SHA512_BLOCK_SIZE * 2];
    uint32_t padding_len;
    uint32_t type; /*hash算法的模式*/
    uint32_t dir;
    uint32_t padding_mode;  /*hash算法的填充模式*/
} crypto_hash_req_ctx_t;
```

3.5 RSA 算法运算接口

- 原型：int do_rsa_crypto(crypto_rsa_req_ctx_t *req_ctx)
- 作用: 主要实现对 RSA 算法进行加解密
- 参数：
  - req_ctx: 为 RSA 算法上下文的结构体
- 返回值：
  - 0：成功
  - 负数：失败
- 上下文数据结构：

```
typedef struct {
    uint8_t *key_n;  /*公钥模数*/
    uint32_t n_len;
    uint8_t *key_e;  /*公钥指数*/
    uint32_t e_len;
    uint8_t *key_d; /*私钥*/
    uint32_t d_len;
    uint8_t *src_buffer;
    uint32_t src_length;
    uint8_t *dst_buffer;
    uint32_t dst_length;
    uint32_t dir;  /*0--加密，1--解密*/
    uint32_t type; /*RSA算法的模式*/
    uint32_t bitwidth;  /*RSA算法位宽*/
} crypto_rsa_req_ctx_t;
```

3.6 RNG 算法运算接口

- 原型：int do_rng_gen(crypto_rsa_req_ctx_t *req_ctx)
- 作用: 主要实现随机数的生成
- 参数：
  - req_ctx: 为 RNG 算法上下文的结构体
- 返回值：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 0：成功
- 负数：失败

- 上下文数据结构：

```
typedef struct {
    uint8_t *rng_buf;
    uint32_t rng_len;
    uint32_t mode;
    uint8_t *key;
    uint32_t key_len;
} crypto_rng_req_ctx_t;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 hwcrypto 设备

由于硬件接口不一，种类繁杂，软件对接起来比较困难，RT-Thread 推出了 hwcrypto 硬件加解密驱动框架，并对接了常见的安全传输套件。

sunxi 已经适配 RT-Thread 的 hwcrypto 硬件加解密驱动框架 (目前只适配 AES 算法)，因此可以直接使用 hwcrypto 的 API 接口即可。

由于 hwcrypto 的框架的接口众多，这里只选择常用 AES 算法的接口介绍，其他接口可以参考 RT-Thread 官网。

说明

hwcrypto 设备仅在 Melis 下支持，FreeRTOS 系统不支持。

4.1 hwcrypto 配置介绍

其 menuconfig 的配置如下：

打开配置CONFIG_DRIVER_CRYPTO_ENGINE，支持 hwcrypto 打开CONFIG_SUNXI_CE_TEST即可在命令行执行对应命令测试对应算法

```
Kernel Setup  --->
 Drivers Setup --->
  Melis Source Support --->
   [*] Support CE Driver
```

```
Kernel Setup  --->
 Drivers Setup --->
  Drivers Test Sample1 --->
   [*] Ce Test
```

4.2 symmetric 设备管理接口

4.2.1 创建对称加解密上下文

- 原型：struct rt_hwcrypto_ctx rt_hwcrypto_symmetric_create(struct rt_hwcrypto_device device, hwcrypto_type type);
- 作用：对称加解密设备的句柄，创建对称加解密上下文
- 参数：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - device: 加解密设备句柄，选择 rt_hwcrypto_dev_default() 即可
  - type: 对称加解密算法模式
- 返回值：
  - NULL: 失败
  - 非空：成功

4.2.2 设置加解密密钥

- 原型：rt_hwcrypto_symmetric_setkey(ctx, key, keybits);
- 作用：根据对称加解密的上下文、密钥与密钥及长度，设置加解密密钥
- 参数：
  - ctx: 上下文
  - key: 输入密钥
  - keybits: 密钥长度
- 返回值：
  - RT_EOK：成功
  - 其他：失败

4.2.3 设置对称加解密初始化向量

- 原型：rt_err_t rt_hwcrypto_symmetric_setiv(struct rt_hwcrypto_ctx ctx, const rt_uint8_t iv, rt_size_t len);
- 作用：根据对称加解密的上下文、初始化向量与向量长度，设置对称加解密初始化向量
- 参数：
  - ctx: 上下文
  - iv: 初始化向量
  - len: 初始化向量长度
- 返回值：
  - RT_EOK: 成功
  - 其他: 失败

4.2.4 加解密接口

- 原型：rt_err_t rt_hwcrypto_symmetric_crypt(struct rt_hwcrypto_ctx ctx, hwcrypto_mode mode, rt_size_t length, const rt_uint8_t in, rt_uint8_t *out);
- 作用：对数据进行加解密

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 参数：
  - ctx: 上下文
  - mode: 加密或解密
  - length: 数据长度，接口规定，输入长度必须 16byte 对齐
  - in: 输入数据
  - out: 输出数据
- 返回值：
  - RT_EOK: 成功
  - 其他: 失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 模块使用范例

1. 对于 FreeRTOS，可参考驱动 APIs 测试代码（lichee/rtos-hal/hal/test/ce/test_ce.c）。

命令	功能
hal_ce aes	测试 AES 算法，包括 ecb/cbc/ctr/ofb/cfb8 模式，key 长度包括（128/192/256bit）
hal_ce aes_sel_key	测试 AES 算法，ecb 模式下，ssk/huk/rssk，key 长度为 256bit
hal_ce des	测试 DES 算法，包括 ecb/cbc/ctr 模式，key 长度包括（128/192/256bit）
hal_ce hash	测试 HASH 算法，包括 md5/sha1/sha224/sha256/sha384/sha512 模式
hal_ce rsa	测试 RSA 算法，包括 512/1024/2048bit
hal_ce rng	测试 RNG 算法，包括 TRNG/PRNG
hal_ce all	同时测试上述所有算法

2. 对于 Melis，可参考驱动 APIs 测试代码（source/ekernel/drivers/test/test_hwcrypt.c），提供 hwcrypt 对接 CE 驱动的 demo。

命令	功能
ce_test aes	测试 AES 算法，包括 ecb/cbc/模式，key 长度包括（128/192/256bit）
ce_test aes_sel_key	测试 AES 算法，ecb/cbc 模式下，ssk/huk/rssk，key 长度为 256bit

- 使用 SSK、HUK、RSSK 进行加解密的示例

在我们的测试示例中，我们通过指定 KEY_SEL_SSK、KEY_SEL_HUK 或 KEY_SEL_RSSK 字符串来选择对应的密钥进行加解密操作。如果系统中已经烧录了相应的 SSK、HUK 或 RSSK，那么在执行加解密时，系统会自动获取这些真实的密钥进行计算。反之，如果这些密钥尚未烧录，系统则会使用全零的密钥进行计算。

1）**准备密钥**首先，确保你的系统已经烧录了所需的密钥。如果密钥尚未烧录，系统将默认使用全零密钥。

2）**选择密钥**在代码中，通过设置 key 参数为 KEY_SEL_SSK、KEY_SEL_HUK 或 KEY_SEL_RSSK 来指定使用哪种密钥（示例中以十六进制的形式输入）。

KEY_SEL_SSK、KEY_SEL_HUK 和 KEY_SEL_RSSK十六进制形式如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
__aligned(64) uint8_t aeskey_ssk[12] = {
  0x4b, 0x45, 0x59, 0x5f, 0x53, 0x45, 0x4c, 0x5f, 0x53, 0x53, 0x4b, 0x00
};
__aligned(64) uint8_t aeskey_huk[12] = {
  0x4b, 0x45, 0x59, 0x5f, 0x53, 0x45, 0x4c, 0x5f, 0x48, 0x55, 0x4b, 0x00
};
__aligned(64) uint8_t aeskey_rssk[13] = {
  0x4b, 0x45, 0x59, 0x5f, 0x53, 0x45, 0x4c, 0x5f, 0x52, 0x53, 0x53, 0x4b, 0x00
};

__aligned(64) uint8_t *aes_selkey[3] = {aeskey_ssk, aeskey_huk, aeskey_rssk};
```

密钥参数设置如下：

```
rt_hwcrypto_symmetric_setkey(ctx, aes_selkey[j], keybits);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明


、 
、全志科技、  （不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。